

CSCE 701 CYBERSECURITY · MODULE 02 · LECTURE NOTES

Authentication

Proving who you are, and the ways it actually fails.

MODULE OUTCOMES

- Explain the three factors and what each resists.
- Explain password storage correctly and the reasoning behind it.
- Explain why the old password rules were wrong.
- Compare second factors by what attack each stops.
- Design account recovery, which is the hard part.

1 The factors

Factor	What it resists	What it fails against
Something you know (a password, a PIN)	Casual impersonation by someone who does not know it.	Phishing, reuse across sites, database breach, keylogging, and shoulder-surfing. Every one of which happens at scale.
Something you have (a security key, a device)	Remote attacks entirely, if the factor is cryptographically bound to the origin — see the callout.	Physical theft, and the SMS-based version fails against SIM swapping, which is a routine attack rather than an exotic one.
Something you are (a fingerprint, a face)	Casual access to an already-unlocked device.	It cannot be revoked, and it is usually a local unlock rather than authentication to a remote service — the device authenticates, and the biometric unlocks the device's key.

The biometric row is the one that is most often misunderstood: a fingerprint is a convenient local unlock and not a secret, because you cannot change it after a compromise and you leave copies of it on every surface you touch. Which means it is a reasonable way to unlock a key stored in a device's secure element, and a poor thing to send to a server. The distinction between 'unlocks a local credential' and 'is a credential' is the one to hold.

PHISHING RESISTANCE IS THE PROPERTY THAT MATTERS

A second factor that the user can read out, type in, or approve can be relayed by a phishing site — which asks the user for it and forwards it to the real service within seconds.

So SMS codes, authenticator app codes, and push approvals are all phishable, and real-time relay phishing toolkits are commodity rather than bespoke — which means the attack is available to any adversary, not only a capable one.

A factor cryptographically bound to the origin is not phishable. WebAuthn and FIDO2 security keys sign a challenge that includes the origin the browser is actually talking to, so a signature produced for the phishing site is cryptographically useless at the real one — the relay has nothing to relay.

Which is the single most important distinction in authentication. 'Two-factor authentication' is not a security level, and origin-bound against relayable is the line that actually matters — an account with SMS second factor and an account with a hardware key are protected against different adversaries, and only one of them is protected against the common one.

2 Password storage

STORE: a salted hash produced by a MEMORY-HARD algorithm.
Argon2id, scrypt, or bcrypt. NOT SHA-256.

WHY EACH PIECE IS THERE:

HASH	so that reading the database does not yield the passwords themselves
SALT	unique per user, so that one precomputed table cannot crack many accounts at once, and so that two users with the same password do not have the same stored value
SLOW	so that guessing costs the attacker real time. SHA-256 is FAST, which is exactly wrong here: a GPU computes billions of SHA-256 hashes per second.
MEMORY-HARD	so that specialised hardware (GPUs, FPGAs, ASICs) loses its advantage, because memory bandwidth does not scale the way raw arithmetic does

AND: a PEPPER -- a secret key held separately from the database and mixed into the hash -- means that a database dump ALONE is not crackable at all. Underused, and cheap.

WHAT NOT TO DO

- your own scheme, ever
- encryption instead of hashing -- a decryptable store means that someone, somewhere, can decrypt it
- a single global salt, which is not a salt
- and truncating or case-normalising the password before hashing, which quietly shrinks the search space

THE OLD PASSWORD RULES WERE WRONG, AND THE EVIDENCE CHANGED THEM

Composition requirements (one uppercase, one digit, one symbol) and forced periodic rotation produced predictable passwords — Password1! becoming Password2! at the next rotation — and **drove** reuse across sites and writing passwords down.

So the current guidance reverses them: a long minimum length, no composition rules, no forced expiry without evidence of compromise, and a check of the proposed password against lists of known-breached passwords. NIST SP 800-63B says this explicitly.

Which is a human-factors result rather than a cryptographic one (Module 11) — the old rules maximised theoretical entropy and minimised actual entropy, because people are highly predictable when constrained, and the constraint collapsed the distribution rather than widening it.

And it is worth knowing as an example of security guidance being corrected by measurement — which happens rarely and should happen far more often (Module 12's measurement problem). The rules persisted for two decades on the strength of an argument that was never tested.

3 Sessions

- **A high-entropy, unpredictable session identifier**, generated from a cryptographic random source — which is CSCE 658 Module 13 §2's **weak-generator failure in its single most common real-world form**, and it has caused account-takeover vulnerabilities repeatedly.
- **Transmitted and stored safely**: the Secure flag so it is never sent over plaintext, HttpOnly so script cannot read it, and SameSite to limit cross-site submission (Module 06 §3).
- **Rotated on any privilege change. Failing to issue a fresh identifier at login enables session fixation** — an attacker sets a known session identifier in the victim's browser, the victim logs in, and the attacker now holds an authenticated session.
- **Revocable server-side. A self-contained stateless token that cannot be revoked is a credential with a fixed lifetime**, which is a **real trade and not a free one** — convenient to validate, and impossible to withdraw when an account is compromised or an employee leaves. **So either keep the lifetime short or keep server-side state**, and decide which deliberately.
- **And bounded in lifetime**, with idle and absolute timeouts chosen from the threat model (a banking session and a news site's session are not the same problem) rather than from habit or from a framework default.

4 Recovery

ACCOUNT RECOVERY IS THE WEAKEST LINK IN MOST AUTHENTICATION SYSTEMS

Strong authentication with weak recovery is weak authentication — the attacker simply uses the **recovery path instead of the login path**, which is precisely Module 01 §1's weakest-link observation applied to the most commonly neglected component.

And the classic high-profile account takeovers are almost all recovery failures: security questions whose answers are public record or guessable, SMS-based reset defeated by SIM swapping, email-based reset where the email account was the weaker one, and support staff socially engineered into performing a manual reset.

So design it deliberately rather than as an afterthought: require multiple recovery factors, impose a delay with notification to the existing contact addresses (so the legitimate owner can intervene), and give support staff a documented procedure they cannot be talked around. The notification-and-delay combination is the cheapest effective control.

And accept that it is a genuine trade against lockout. Stronger recovery means more legitimate users permanently locked out of their accounts, which is a real and sometimes severe cost — **and the right point on that trade depends entirely on what the account controls**. A cryptocurrency wallet and a recipe site should not make the same choice.

- **1 · Use a well-maintained identity provider or library. Do not implement authentication yourself** — the failure modes are numerous, known, and subtle, and there is no credit for a bespoke implementation.

- **2 · Offer origin-bound second factors** (WebAuthn or passkeys) **and prefer them in the flow** — offering them and defaulting to SMS leaves the phishable path available (§1).
- **3 · Store passwords correctly** (§2) **and check proposed passwords against breached-password lists**, which addresses the actual attack better than any composition rule.
- **4 · Rate-limit and monitor. Credential stuffing is the most common attack against authentication and it is highly detectable** — many accounts, one source, high failure rate (Module 09).
- **5 · And design recovery as carefully as you design login**, because that is where the attack will go once login is hard. **Credential stuffing is the attack to design against** — it uses passwords breached elsewhere, which is why the breached-list check and the origin-bound factor matter far more than password complexity rules ever did.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
NIST SP 800-63B — Digital Identity Guidelines, Authentication (free)	§2's current guidance , including the reversal of the composition and rotation rules, with the reasoning.
OWASP — Authentication, Password Storage, and Session Management Cheat Sheets (free)	The working reference for every section of this module , kept current.
The WebAuthn specification and passkeys documentation (free)	§1's origin-bound factor — and the origin-binding mechanism is worth reading, because it is what makes the phishing resistance work.
Bonneau et al. — The Quest to Replace Passwords (free)	A systematic comparison of authentication schemes on usability, deployability, and security — and the framework explains why passwords persisted.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Classify the second factors** offered by five services you use as origin-bound or relayable.
- 2 **Set up a hardware security key or passkey** on an account, and note the friction.
- 3 **Hash a password with SHA-256 and with Argon2id** and compare the time per hash.
- 4 **Estimate how many guesses per second** each permits on a GPU, and what that means for a weak password.
- 5 **Implement salted memory-hard password storage** correctly, and then add a pepper.

- 6 **Check a password list against a breached-password service** using its k-anonymity API.
- 7 **Inspect the session cookie flags** on five sites and report which are missing.
- 8 **Test whether a service rotates the session identifier at login** — on your own account.
- 9 **Map the recovery paths** for your own most important account, and identify the weakest.
- 10 **Design a recovery flow** for a system of yours, with the lockout trade stated explicitly.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the three factors and what each resists and fails against.
- 2 Why is a biometric not a secret?
- 3 What property sorts second factors, and why is 'two-factor' not a security level?
- 4 Give the four components of correct password storage and why each is there.
- 5 Why is SHA-256 the wrong choice, and what is a pepper?
- 6 Why were the old password rules wrong, and what replaced them?
- 7 Name five requirements of a session, and the trade in stateless tokens.
- 8 Why is recovery the weakest link, and what are the classic failures?
- 9 What trade does stronger recovery make?
- 10 Give the five-step priority order, and name the attack it is designed against.